



ACTIVIDAD POO

Kevin Santiago Bravo Campo
bravocampok@gmail.com

Contenido

POO: Animal, Propietario y Atención	2
Objetivo	2
Sección 1: Comprendiendo la Clase Animal	2
Sección 3: Comprender this.....	2
Sección 4: Comprender el objeto.....	2
Sección 5: Comprender el método.....	2
Sección 6: Clase Propietario	2
Sección 7	3
Sección 8 Comprender Push.....	3
Sección 9 Seguir los datos	3
Sección 10: Accediendo a objetos en arreglos.....	3
Sección 11 Agregar otra mascota	4
Sección 12	4
Sección 13: Métodos de Arreglos (Banco de palabras)	4
Sección 14	4
Sección 15	4
Sección 16	4
Sección 17	4
Sección 18	5
Sección 19	5
Sección 20 Actividad final – completar el programa.....	5
Sección 21 Crear los objetos.....	6
Sección 22 Reflexión	6

POO: Animal, Propietario y Atención

Objetivo

Comprender cómo funciona un programa desarrollado con clases y objetos en JavaScript y completar las partes faltantes del código.

Sección 1: Comprendiendo la Clase Animal

1. ¿Cuál es el nombre de la clase?

Respuesta: Animal

2. ¿Cuáles son sus atributos?

Respuesta: nombre, raza, especie, edad

3. ¿Para qué sirve el constructor?

Respuesta: Inicializar los atributos del objeto cuando se crea una nueva instancia

4. ¿Qué hace registrarNombre()?

Respuesta: Actualiza/modifica el nombre del animal mediante asignación

5. ¿Qué hace mostrarInformacion()?

Respuesta: Muestra en la consola el nombre y raza del animal concatenados

Sección 3: Comprender this

- this hace referencia a: el objeto actual (la instancia de la clase)
- this.nombre representa: el atributo nombre del objeto actual
- nombre representa: el parámetro recibido en el constructor

Sección 4: Comprender el objeto

Para: let colmillo = new Animal("colmillo", "pitbull", "perro", 12)

- Nombre: colmillo
- Raza: pitbull
- Especie: perro
- Edad:

colmillo es: Un objeto

Sección 5: Comprender el método

Observe: colmillo.mostrarInformacion()

- Objeto que ejecuta el método: colmillo
- Método ejecutado: mostrarInformacion()
- ¿Qué información muestra?: colmillo pitbull

Sección 6: Clase Propietario

```

class Propietario {

    nombrePropietario
    Direccion
    telefono

    constructor(nombre, direccion, telefono) {

        this.nombrePropietario = nombre
        this.direccion = direccion
        this.telefono = telefono

        this.mascota = []
    }
}
complete:

```

- nombrePropietario → almacena el nombre del propietario
- direccion → almacena la dirección del propietario
- telefono → almacena el teléfono del propietario
- mascota → almacena un arreglo de objetos Animal

¿Qué significa this.mascota = []?

Inicializa un arreglo vacío que guardará las mascotas del propietario

¿Por qué mascota es un arreglo?

Porque un propietario puede tener varias mascotas, el arreglo permite almacenar múltiples objetos

Sección 7

mascota es un arreglo porque puede almacenar múltiples elementos.

Sección 8 Comprender Push

push() sirve para: Agregar un elemento al final

Sección 9 Seguir los datos

```

let alexa = new Propietario(
    "Alexa",
    "123",
    "1111"
)

```

```

let colmillo = new Animal(
    "colmillo",
    "pitbull",
    "perro",
    12
)

```

alexa.registrarMascota(colmillo)

- Antes: alexa.mascota = []
- Después: alexa.mascota = [colmillo]
- ¿Qué objeto quedó guardado?: colmillo

Sección 10: Accediendo a objetos en arreglos

alex$\rightarrow$ mascotas $\rightarrow$ [0] $\rightarrow$ nombre
Complete:
alex$\rightarrow$ mascota[0].nombre
¿Qué resultado obtenemos?
Colmillo

Sección 11 Agregar otra mascota

Si agregamos otra mascota (michi):

- alex$\rightarrow$ mascota = [colmillo, michi]
- Michi está en: alex$\rightarrow$ mascota[1]
- Accedemos con: console.log(alex$\rightarrow$ mascota[1].nombre)
- Resultado: Michi

Sección 12

Si Colmillo está en alex$\rightarrow$ mascota[0], ¿en qué posición está Michi?

alex$\rightarrow$ mascota[1]
console.log(alex$\rightarrow$ mascota[1].nombre)
Resultado: "Michi"

Sección 13: Métodos de Arreglos (Banco de palabras)

- Agregar al final: push(mascota)
- Eliminar del final: pop()
- Agregar al inicio: unshift(mascota)
- Eliminar del inicio: shift()

Sección 14

this.fecha = fecha
this.motivo = motivo
this.diagnostico = diagnostico
this.tratamiento = tratamiento

Sección 15

Complete: atencion1. **mostrarInformacion()**

Sección 16

¿Por qué atenciones es un arreglo en Animal?

Porque una mascota puede tener varias atenciones veterinarias a lo largo del tiempo

Sección 17

this.atenciones. **push(nuevaAtencion)**
Después: colmillo.registrarAtencion(**atencion1**)

Sección 18

Expresión: alexa.mascotas[0].atenciones[0].diagnostico

- alexa → Objeto Propietario
- .mascotas → Arreglo de mascotas del propietario
- [0] → Primer elemento del arreglo (Colmillo)
- .atenciones → Arreglo de atenciones de la mascota
- [0] → Primera atención de la mascota
- .diagnostico → Atributo diagnóstico (Gastritis)

Sección 19

- Nombre de Colmillo: console.log(alex.mascotas[0].nombre) → colmillo
- Raza de Colmillo: console.log(alex.mascotas[0].raza) → pitbull
- Diagnóstico: console.log(alex.mascotas[0].atenciones[0].diagnostico) → Gastritis
- Tratamiento: console.log(alex.mascotas[0].atenciones[0].tratamiento) → Dieta especial

Sección 20 Actividad final – completar el programa

```
class Animal {
  constructor(nombre, raza, especie, edad) {
    this.nombre = nombre
    this.raza = raza
    this.especie = especie
    this.edad = edad
    this.atenciones = []
  }

  registrarAtencion(nuevaAtencion) {
    this.atenciones.push(nuevaAtencion)
  }

  mostrarInformacion() {
    console.log(
      this.nombre + " " +
      this.raza + " " +
      this.especie + " " +
      this.edad
    )
  }
}
```

```
class Propietario {
  constructor(nombre, direccion, telefono) {
    this.nombre = nombre
    this.direccion = direccion
    this.telefono = telefono
    this.mascotas = []
  }

  registrarMascota(nuevaMascota) {
    this.mascotas.push(nuevaMascota)
  }

  mostrarInformacion() {
    console.log(
      "Mascota: " +

```

```

        this.mascotas[0].nombre +
        " Propietario: " +
        this.nombre
    )
}
}

class Atencion {
    constructor(fecha, motivo, diagnostico, tratamiento) {
        this.fecha = fecha
        this.motivo = motivo
        this.diagnostico = diagnostico
        this.tratamiento = tratamiento
    }

    mostrarInformacion() {
        console.log("Fecha: " + this.fecha)
        console.log("Motivo: " + this.motivo)
        console.log("Diagnóstico: " + this.diagnostico)
        console.log("Tratamiento: " + this.tratamiento)
    }
}

```

Sección 21 Crear los objetos

```

let colmillo = new Animal(
    "Colmillo",
    "Pitbull",
    "Perro",
    12
)

let alexa = new Propietario(
    "Alexa",
    "Calle 10",
    "3001234567"
)

let atencion1 = new Atencion(
    "11/09/2026",
    "Dolor abdominal",
    "Gastritis",
    "Dieta especial"
)

alexa.registrarMascota(colmillo)
colmillo.registrarAtencion(atencion1)
colmillo.mostrarInformacion()
alexa.mostrarInformacion()
atencion1.mostrarInformacion()

```

Sección 22 Reflexión

1. ¿Por qué mascotas es un arreglo?
Porque un propietario puede tener varias mascotas, por eso se utiliza un arreglo para almacenarlas.
2. ¿Por qué atenciones es un arreglo?
Porque una mascota puede tener varias atenciones veterinarias.
3. ¿Qué significa [0]?
Significa que estamos accediendo al primer elemento de un arreglo.

4. ¿Qué significa .nombre?

Significa que estamos accediendo al atributo nombre de un objeto.

5. Explique qué hace alexa.mascotas[0].nombre:

Accede al arreglo de mascotas de Alexa, selecciona la primera mascota y obtiene su nombre. El resultado es Colmillo.

6. Explique qué hace alexa.mascotas[0].atenciones[0].diagnostico:

Accede a las mascotas de Alexa, selecciona la primera mascota, entra a sus atenciones, selecciona la primera atención y obtiene su diagnóstico. El resultado es Gastritis.

Resultado esperado

La lógica del programa es: Propietario → mascotas[] → mascota[0] → atenciones[] → atención[0] → diagnóstico. Un propietario puede tener varias mascotas, una mascota puede tener varias atenciones y cada atención contiene información como fecha, motivo, diagnóstico y tratamiento.